Send
Flow

TUTORIAL TÉCNICO · NODE.JS

Claude + SendFlow via MCP

Guia completo para criar um servidor MCP local em Node.js e integrar a SendAPI do Sendflow ao Claude Code — sem intermediários, sem código extra.

VERSÃO	FERRAMENTAS	RUNTIME
2.0 — Node.js	41 tools	Node.js 18+

NAVEGAÇÃO

Índice

CONCEITO

01 · O que é o MCP? Pág 03

02 · O que você vai construir Pág 04

03 · Pré-requisitos Pág 05

CRIANDO A INTEGRAÇÃO

04 · Prompt para o Claude Cowork Pág 06

05 · Estrutura do projeto Pág 07

06 · O servidor MCP (index.js) Pág 08

07 · Instalar e registrar Pág 09

USANDO

08 · Testar a integração Pág 10

09 · Uso no dia a dia Pág 11

10 · Troubleshooting Pág 12

11 · Checklist final Pág 13

Sobre este tutorial

Este guia usa **Node.js** — uma alternativa mais simples ao servidor em Python. Não é necessário criar ambientes virtuais, instalar Python ou configurar dependências complexas. Tudo funciona com um único `npm install`.

O que é o MCP?

O **Model Context Protocol (MCP)** é um padrão aberto criado pela Anthropic que permite ao Claude se conectar a ferramentas e serviços externos de forma segura e padronizada. Pense nele como uma "tomada universal" entre o Claude e qualquer API ou sistema que você quiser integrar.

Sem o MCP, o Claude não consegue chamar APIs externas diretamente. Com o MCP, você cria um servidor local que expõe as capacidades de uma API como **ferramentas** — e o Claude aprende a usá-las automaticamente a partir de linguagem natural.

> **Em outras palavras:**
> Você diz "liste minhas campanhas", o Claude entende que deve chamar a ferramenta `list_campaigns`, o servidor MCP monta a requisição HTTP autenticada e devolve o resultado — tudo sem você escrever uma linha de código na hora.

Como funciona na prática

1. **Você cria um servidor MCP local**
   Um arquivo Node.js que declara ferramentas e define como cada uma chama a API do Sendflow.

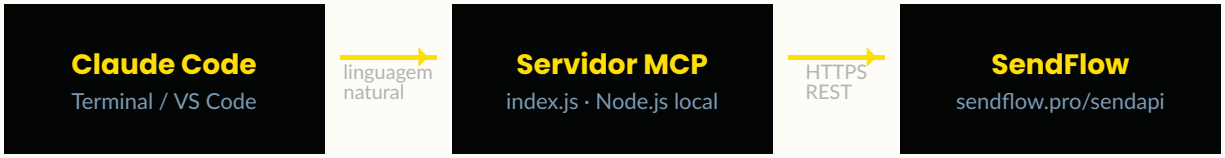
2. **O Claude Code carrega o servidor**
   Ao iniciar, o Claude Code descobre as 41 ferramentas disponíveis e aprende seus parâmetros automaticamente.

3. **Você conversa em linguagem natural**
   O Claude escolhe a ferramenta certa, monta a chamada, executa e devolve a resposta formatada para você.

> **Por que não usar a API diretamente?**
> Você pode — mas com o MCP você não precisa lembrar endpoints, parâmetros ou formatos de data. O Claude cuida disso. Você foca no que quer fazer, não em como fazer.

O que você vai construir

Um servidor MCP local em Node.js que conecta o Claude Code à SendAPI do Sendflow, expondo **41 ferramenta** organizadas em 8 categorias.



As 41 ferramentas disponíveis

10 Campanhas

- `list_campaigns`
- `get_campaign`
- `create_campaign`
- `update_campaign`
- `delete_campaign`
- `get_campaign_analytics`
- `get_campaign_groups`
- `update_campaign_redirect_link`
- `generate_campaign_leadscoring`
- `get_campaign_leadscoring_url`

9 Ações

- `send_text_to_campaign`
- `send_image_to_campaign`
- `send_video_to_campaign`
- `send_audio_to_campaign`
- `send_message_universal`
- `create_whatsapp_group`
- `make_group_admin`
- `analyze_groups`
- `find_participant`

7 Contas

- `list_accounts`
- `create_account`
- `update_account`
- `delete_account`
- `connect_account`
- `disconnect_account`
- `get_account_qrcode`

4 Mensagens Diretas

- `send_direct_text`
- `send_direct_image`
- `send_direct_video`
- `send_direct_audio`

Pré-requisitos

Você precisa de 4 itens instalados antes de começar. Verifique cada um.

⬡

Node.js 18 ou superior

O runtime JavaScript que executa o servidor MCP localmente.

```
node --version
```

Deve retornar v18.x ou superior.
Download: nodejs.org

◈

Claude Code (CLI)

A interface de linha de comando do Claude que usa os servidores MCP.

```
claude --version
```

Se não tiver:
`npm install -g @anthropic-ai/claude-code`

◆

API Key do Sendflow

Chave de autenticação no formato `send_api-...`

```
Painel Sendflow → API Keys
```

Guarde em um lugar seguro. Nunca versione no Git.

△

Git for Windows

Obrigatório no Windows para o Claude Code funcionar corretamente.

```
git --version
```

Download: git-scm.com/downloads/win

Configurar o Git Bash no Windows (uma vez só)

Abra o PowerShell como administrador e execute:

`[System.Environment]::SetEnvironmentVariable('CLAUDE_CODE_GIT_BASH_PATH', 'C:\Program Files\Git\bin\bash.exe', 'Machine')`

Depois feche e reabra o terminal.

Login no Claude Code

Após instalar, autentique uma vez:

`claude login`

Um navegador vai abrir para confirmar. Depois disso o login fica salvo.

Prompt para o Claude Cowork

Se preferir não criar os arquivos manualmente, use o **Claude Cowork** para gerar toda a estrutura de uma vez. Cole o prompt abaixo em uma nova conversa, anexe o arquivo de documentação da API, e o Claude criará todos os arquivos automaticamente.

PROMPT

Preciso criar um servidor MCP local em Node.js para integrar a API do Sendflow com o Claude Code.

O que preciso que você crie na pasta C:\Users\[SEU_NOME]\Desktop\sendapi**:**

1. index.js — servidor MCP com 41 ferramentas cobrindo toda a SendAPI
2. package.json — com @modelcontextprotocol/sdk v1.9.0, type: "module"
3. .gitignore — ignorando node_modules/ e .env
4. .env.example — template com SENDFLOW_API_KEY=send_api-...
5. SETUP.md — instruções de instalação e registro

Requisitos técnicos:
- Node.js com ESM modules ("type": "module")
- A API key deve ser lida de process.env.SENDFLOW_API_KEY
- Base URL: https://sendflow.pro/sendapi
- Auth: Authorization: Bearer {KEY}

As 41 ferramentas devem cobrir (veja a documentação em anexo):
- Campanhas (/releases) — 10 tools
- Grupos de campanha (/release-groups) — 4 tools
- Acoes (/actions) — 9 tools
- Mensagens diretas (/send-*) — 4 tools
- Templates (/message-templates) — 4 tools
- Contas (/accounts) — 7 tools
- Numeros bloqueados (/block-numbers) — 2 tools
- Verificacao (/verify-number) — 1 tool

Apos criar os arquivos, me da os comandos para instalar e registrar no Claude Code.

O que anexar no Cowork

Antes de enviar o prompt, anexe o arquivo `sendflow-api-documentacao-completa.md` com todos os endpoints. O Claude usará a documentação para criar as ferramentas corretas. Se não tiver o arquivo, peça ao time do Sendflow ou gere via painel da API.

Estrutura do projeto

O servidor MCP é composto por apenas **5 arquivos**. Estrutura simples, sem configuração complexa.

ESTRUTURA DA PASTA

```
sendapi/
├── index.js          # servidor MCP — 41 ferramentas (~500 linhas)
├── package.json      # dependências do projeto
├── .env.example      # template da API key (nunca commite o .env real)
├── .gitignore        # ignora node_modules/ e .env
├── SETUP.md          # este guia de instalação
└── node_modules/     # criado pelo npm install (não editar)
```

PACKAGE.JSON

```
{
  "name": "sendflow-mcp",
  "version": "2.0.0",
  "type": "module",            // ESM — obrigatório
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "@modelcontextprotocol/sdk": "^1.9.0"
  }
}
```

Por que v1.9.0 e nao a mais recente?
A partir da v1.10.x, o SDK passou a usar zod v4, que tem incompatibilidades de importacao com Node.js v22 em ambientes ESM. A v1.9.0 e estavel e cobre todos os recursos necessarios.

.ENV.EXAMPLE

```
# Copie para .env e preencha com sua chave real
SENDFLOW_API_KEY=send_api-coloque_sua_chave_aqui
```

O servidor MCP (index.js)

O `index.js` é o coração da integração. Ele usa o SDK do MCP para expor as ferramentas da SendAPI ao Claude Code. O arquivo tem ~500 linhas e está pronto para usar — não é necessário editá-lo.

ESTRUTURA DO INDEX.JS — VISÃO GERAL

```
// 1. Importações do SDK MCP
import { Server } from "@modelcontextprotocol/sdk/server/index.js"
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js"

// 2. Configuração — lê a API key do ambiente
const API_KEY = process.env.SENDFLOW_API_KEY
const BASE_URL = "https://sendflow.pro/sendapi"

// 3. Helper HTTP — faz as chamadas autenticadas
async function callAPI(method, path, body) { ... }

// 4. Definição das 41 ferramentas (TOOLS array)
const TOOLS = [
  // Campanhas (10), Grupos (4), Acoes (9), Mensagens diretas (4),
  // Templates (4), Contas (7), Bloqueios (2), Verificacao (1)
]

// 5. Handler — executa a ferramenta certa quando o Claude chama
async function handleTool(name, args) {
  switch (name) { ... 41 cases ... }
}

// 6. Inicialização do servidor via stdio
const server = new Server({ name: "sendflow-mcp", version: "2.0.0" }, ...)
await server.connect(new StdioServerTransport())
```

Cada ferramenta tem 3 partes:

name

— identificador único (ex: `list_campaigns`) ·

description

— o Claude usa isso para saber quando chamar a ferramenta ·

inputSchema

— os parametros esperados (o Claude monta automaticamente)

O servidor roda via stdio

O Claude Code inicia o processo `node index.js` automaticamente e se comunica via stdin/stdout (stdio). Voce nao precisa deixar nenhum servidor rodando em background — o Claude Code gerencia o ciclo de vida do processo.

Instalar e registrar

Com os arquivos criados, sao apenas **3 comandos** para colocar tudo no ar.

1. **Instalar as dependencias**
   Abra o terminal na pasta `sendapi` e execute:

```
cd "C:\Users\raiss\Desktop\sendapi"
npm install
```

2. **Registrar o servidor no Claude Code**
   O flag `--scope user` torna o servidor disponivel em qualquer pasta do seu computador.

```
claude mcp add sendflow --scope user \
  --env SENDFLOW_API_KEY=send_api-SUA_CHAVE_AQUI \
  -- node "C:\Users\raiss\Desktop\sendapi\index.js"
```

3. **Confirmar que foi registrado**
   Deve aparecer `sendflow` na lista:

```
claude mcp list
# sendflow   node C:\Users\raiss\Desktop\sendapi\index.js
```

Precisa substituir o servidor?
Se precisar atualizar o caminho ou a API key:
`claude mcp remove sendflow` — e depois adicione novamente com o comando do passo 2.

Alternativa: Claude Desktop
Se preferir usar o Claude Desktop, adicione o bloco abaixo ao arquivo `%APPDATA%\Claude\claude_desktop_config.json` :

```
{ "mcpServers": { "sendflow": { "command": "node", "args": ["C:\\...\\index.js"],
"env": { "SENDFLOW_API_KEY": "..." } } } }
```

Testar a integracao

Com o servidor registrado, abra o Claude Code no terminal e execute os testes abaixo em ordem. O primeiro teste nao dispara nada — e seguro para comecar.

ABRIR O CLAUDE CODE

```
claude
```

1 **Teste de leitura — lista de contas**
Nao dispara mensagem. So lê dados. Confirme com `y` quando solicitado.

```
Liste minhas contas de WhatsApp.
```

Resultado esperado
O Claude chama `list_accounts` e retorna um JSON com suas contas. Se isso funcionar, a integracao esta ativa.

2 **Teste de campanha**
Lista todas as campanhas com seus IDs.

```
Liste minhas campanhas do Sendflow.
```

3 **Teste de envio real**
Substitua os IDs reais obtidos nos testes anteriores.

```
Use a ferramenta sendflow para enviar o texto
"Teste integracao Claude + SendAPI"
para a campanha RELEASE_ID, usando a conta ACCOUNT_ID.
```

Importante sobre o Claude Desktop vs Claude Code
O Claude Desktop roda o servidor MCP em ambiente sandbox com restricoes de rede. Para chamadas reais a SendAPI, use sempre o
Claude Code (CLI)
via terminal.

Uso no dia a dia

A partir daqui e tudo linguagem natural. O Claude escolhe a ferramenta certa das 41 disponiveis e cuida dos detalhes tecnicos. Exemplos do que voce pode fazer:

"

Me mostra o analytics das minhas ultimas 3 campanhas e qual teve mais cliques hoje.

→ `get_campaign_analytics`

"

Agende uma mensagem 'Promocao relampago!' para a campanha X amanha as 10h com velocidade normal.

→ `send_text_to_campaign`

"

Verifique se os numeros 11999999999 e 11888888888 estao bloqueados para a campanha Y.

→ `verify_number`

"

Crie uma campanha 'Black Friday 2026' e adicione o grupo 120363292004848696@g.us nela.

→ `create_campaign + add_release_group`

"

Bloqueie o numero 5511999999999 como 'Spammer' e confirme o bloqueio.

→ `block_number + list_blocked_numbers`

"

Me mostra o QR code da conta ACC123 — preciso reconectar o WhatsApp.

→ `connect_account + get_account_qrcode`

Dica: combine com arquivos
O Claude Code pode combinar as ferramentas do Sendflow com leitura e escrita de arquivos. Por exemplo: "Leia a planilha contatos.xlsx, verifique quais numeros estao bloqueados na campanha X e me gere um relatorio." O Claude orquestra tudo automaticamente.

Troubleshooting

`SENDFLOW_API_KEY nao definida`

O servidor subiu mas nao encontrou a API key.

Confirme que o `--env SENDFLOW_API_KEY=...` foi passado no `claude mcp add`. A key no parametro `--env` tem precedencia sobre o arquivo `.env`.

`401 Unauthorized`

API Key invalida, expirada ou sem permissao.

Gere uma nova chave no painel do Sendflow. Remova o servidor (`claude mcp remove sendflow`) e registre novamente com a nova chave.

`MCP server sendflow already exists`

O servidor ja estava registrado de uma sessao anterior.

Apenas prossiga — a mensagem e informativa, nao e um erro. Ou remova e adicione novamente: `claude mcp remove sendflow`.

`claude nao e reconhecido / nao e um comando`

O PATH do npm global nao esta no ambiente.

Execute `npm config get prefix` para descobrir onde o npm instalou. Adicione esse caminho ao PATH nas variaveis de ambiente do Windows.

`Claude Code on Windows requires git-bash`

Git for Windows nao instalado ou variavel nao definida.

Instale o Git em git-scm.com e defina a variavel de ambiente `CLAUDE_CODE_GIT_BASH_PATH` apontando para `C:\Program Files\Git\bin\bash.exe`.

`ERR_MODULE_NOT_FOUND ou zod/v4 import error`

Versao incompativel do SDK com Node.js v22.

Garanta que o `package.json` usa `"@modelcontextprotocol/sdk": "^1.9.0"` e rode `npm install` novamente.

Checklist final

Se todos os itens abaixo estiverem marcados, sua integracao Claude + SendFlow esta pronta.

- [ ] `node --version` retorna v18.x ou superior
- [ ] `claude --version` responde com numero de versao
- [ ] Git for Windows instalado e variavel `CLAUDE_CODE_GIT_BASH_PATH` definida
- [ ] Pasta `sendapi/` criada com os arquivos: `index.js`, `package.json`, `.gitignore`, `.env.example`
- [ ] `npm install` rodou sem erros na pasta `sendapi/`
- [ ] `claude mcp add sendflow` executado com a API key correta
- [ ] `claude mcp list` mostra `sendflow` na lista
- [ ] Dentro do `claude`, o prompt "liste minhas contas de WhatsApp" retorna um JSON da SendAPI
- [ ] Dentro do `claude`, o prompt "liste minhas campanhas" retorna dados reais
- [ ] Primeiro envio de teste realizado com sucesso

Sua integracao esta pronta.

A partir daqui, qualquer coisa que voce precisar fazer na SendAPI pode ser pedida em linguagem natural diretamente no terminal. O Claude cuida dos endpoints, parametros e autenticacao.